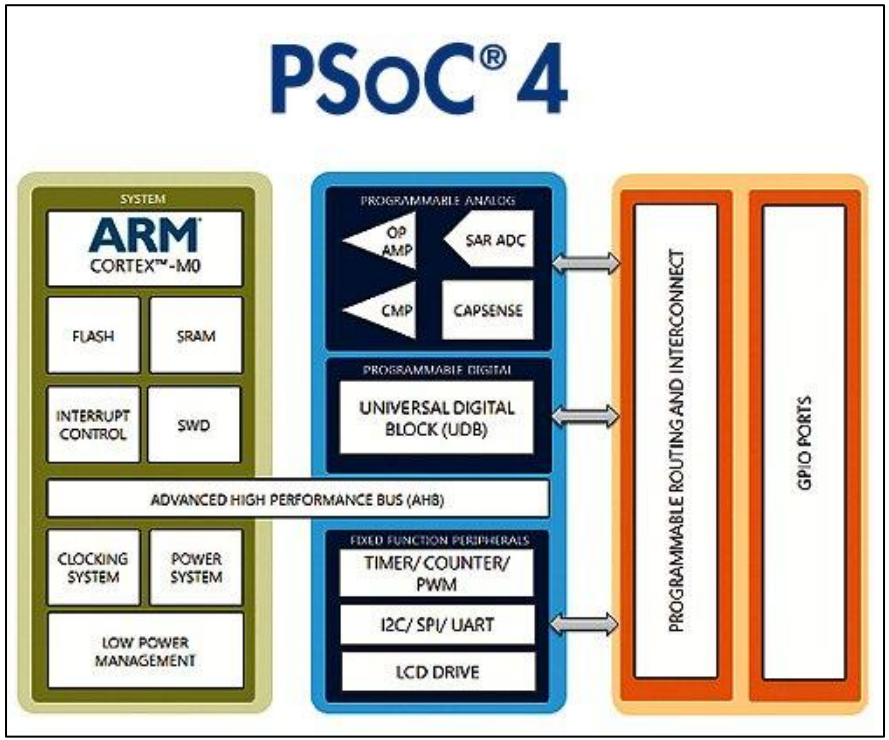
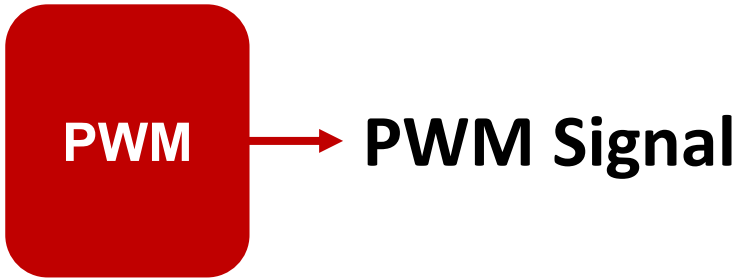
Digital Blocks

PWM

PWM



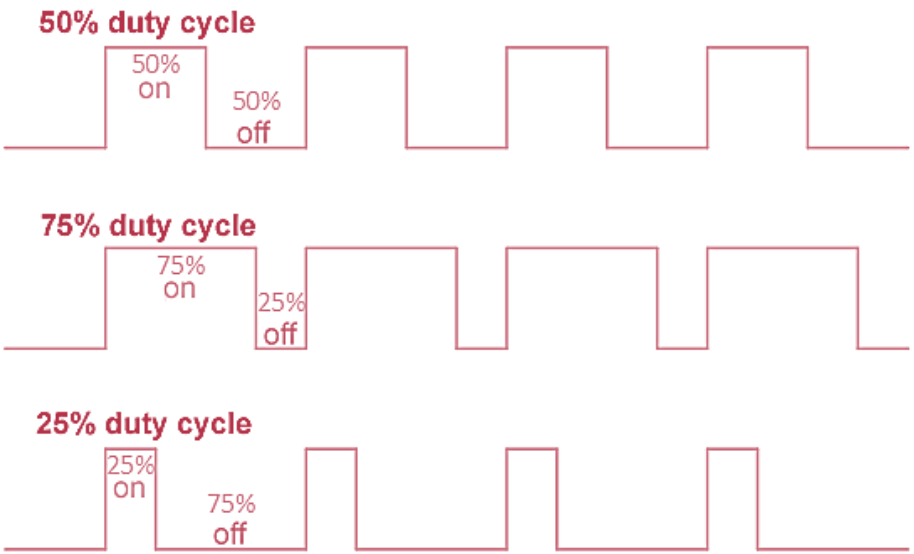
Pulse Width Modulator



https://community.element14.com/products/devtools/technicallibrary/w/documents/11058/cypress-psoc-4-system-on-chip-overview

PWM Signal

"A Square Wave Signal with a Specified Duty Cycle"



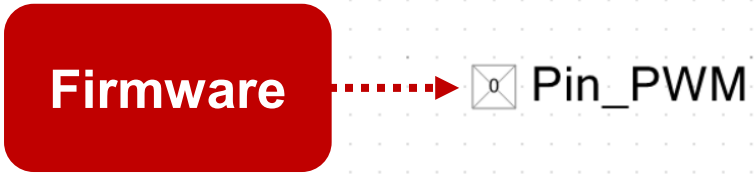
$$\text{Duty Cycle} = \frac{T_{ON}}{T_{ON} + TOFF}$$

$$\text{Freq} = \frac{1}{T_{PERIOD}} = \frac{1}{T_{ON} + TOFF}$$

$$\text{Amplitude} = V_{HIGH} - V_{LOW}$$

https://en.wikipedia.org/wiki/Pulse-width_modulation

Generation (Method #1)



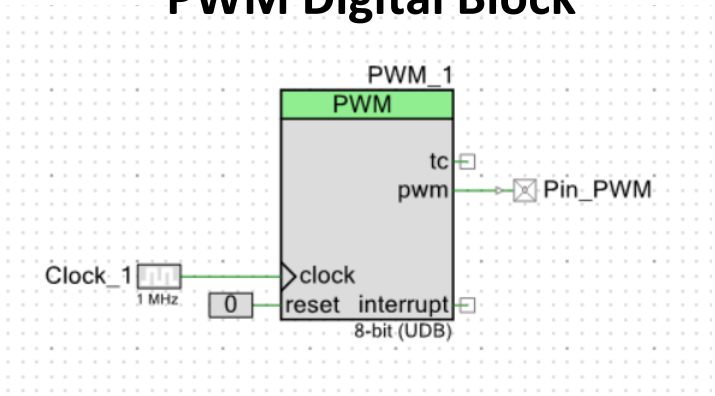

```c
#include "project.h"

#define PERIOD_COUNT 1000
#define DUTY_COUNT 750

int main(void)
{
    CyGlobalIntEnable; /* Enable global interrupts. */

    /* Set the Pin_PWM pin as an output pin */
    Pin_PWM_SetDriveMode(Pin_PWM_DM_STRONG);
    Pin_PWM_Write(0);

    for(;;)
    {
        /* Turn the Pin_PWM pin on for DUTY_COUNT cycles */
        Pin_PWM_Write(1);
        CyDelay(DUTY_COUNT);

        /* Turn the Pin_PWM pin off
        for (PERIOD_COUNT - DUTY_COUNT) cycles */
        Pin_PWM_Write(0);
        CyDelay(PERIOD_COUNT - DUTY_COUNT);
    }
}
```

https://en.wikipedia.org/wiki/Pulse-width_modulation

1kHz PWM signal with a 75% duty cycle

Generation (Method #2)

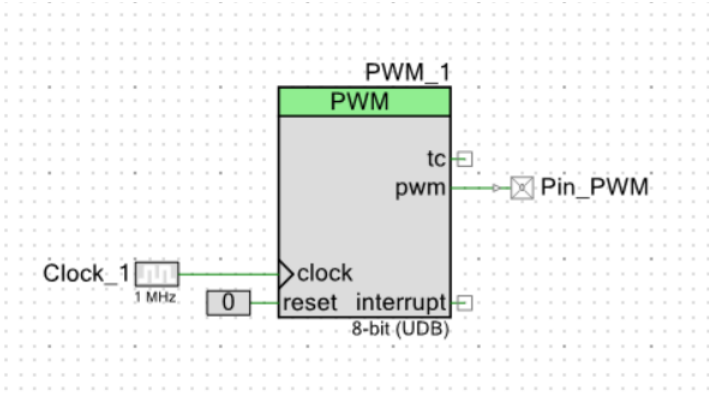
PWM Digital Block




```
#include "project.h"

int main(void)
{
    CyGlobalIntEnable; /* Enable global interrupts. */

    /* Start the PWM component */
    PWM_1_Start();

    /* Set the PWM period to 1ms (1KHz) */
    PWM_1_WritePeriod(1000);

    /* Set the PWM duty cycle to 75% */
    PWM_1_WriteCompare(750);

    for(;;)
    {
        /* Do nothing, the PWM runs in the background */
    }
}
```

https://en.wikipedia.org/wiki/Pulse-width_modulation

Component Settings



PWM Bits = 8bit (or 16bit)

PWM Period = 0-255 (or 0 to 65535)

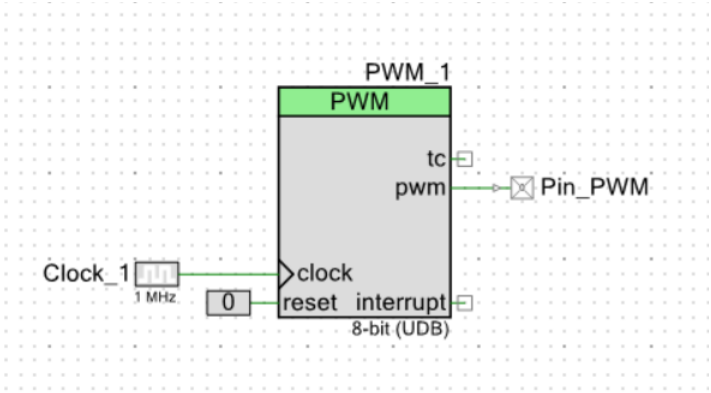
PWM Compare = 0-255 (or 0 to 65535)

$$\text{Freq} = \frac{1}{T_{PERIOD}} = \frac{1}{\text{Period} * \text{Tclock}_{\_1}}$$

$$\text{Duty Cycle} = \frac{\text{Compare}}{\text{Period}}$$

https://en.wikipedia.org/wiki/Pulse-width_modulation

E.g., Settings



PWM Bits = 8bit

PWM Period = 200

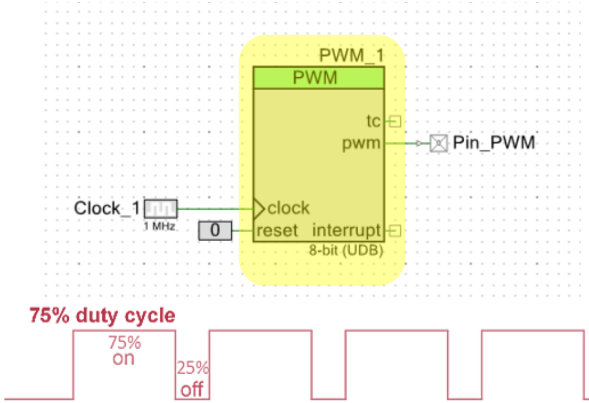
PWM Compare = 50

$$\text{Freq} = \frac{1}{T_{PERIOD}} = \frac{1}{\text{Period} * \text{Tclock}_1} \quad ?$$

$$\text{Duty Cycle} = \frac{\text{Compare}}{\text{Period}} \quad ?$$

https://en.wikipedia.org/wiki/Pulse-width_modulation

Verilog 8bit PWM



PWM Bits = 8bit

PWM Period = 0-255

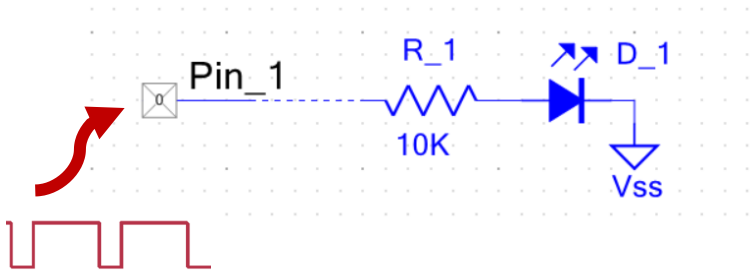
PWM Compare = 0-255

https://en.wikipedia.org/wiki/Pulse-width_modulation

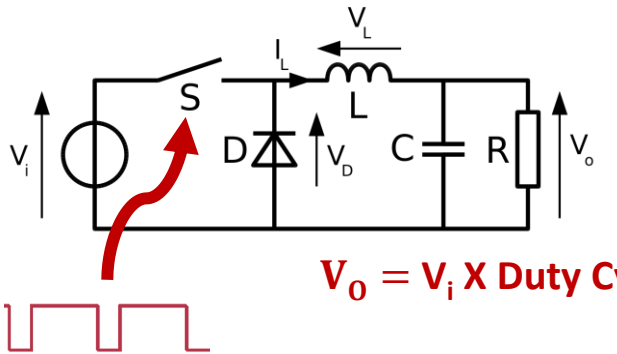
```
module pwm_8bit(
  input clock,
  input reset,
  input [7:0] pwm_compare,
  input [7:0] pwm_period,
  output pwm
);
  reg [7:0] count = 0;
  reg pwm_temp = 1'b0;

  always @(posedge clock) begin
    if (reset) begin
      count <= 0;
    end else begin
      count <= (count == pwm_period) ? 0 : count + 1;
    end
  end

  always @(posedge clock) begin
    if (count < pwm_compare) begin
      pwm_temp <= 1'b1;
    end else begin
      pwm_temp <= 1'b0;
    end
  end

  assign pwm = pwm_temp;

endmodule
```

Applications

a. LED Brightness Ctrl



b. SMPS



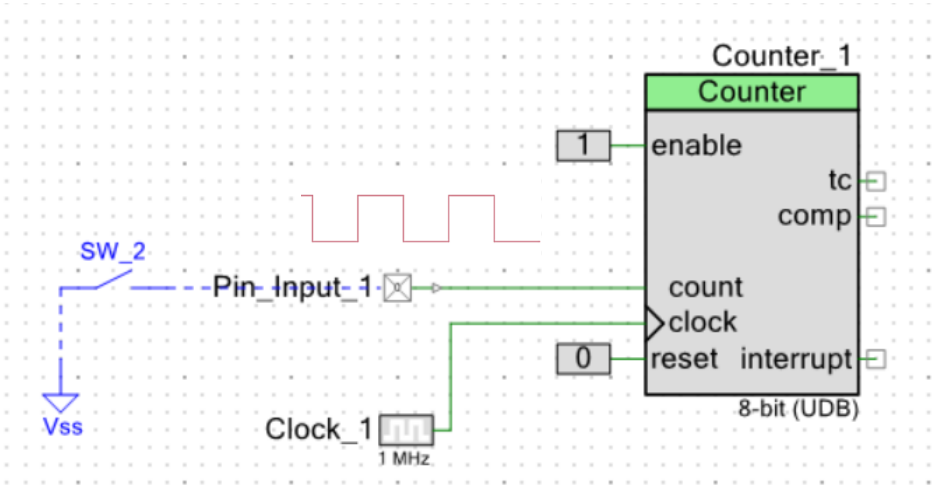
$V_o = V_i$ X Duty Cycle

https://en.wikipedia.org/wiki/Pulse-width_modulation

Counter

Counter

“Used for Counting Events”



Bits = 8bit, 16bit …

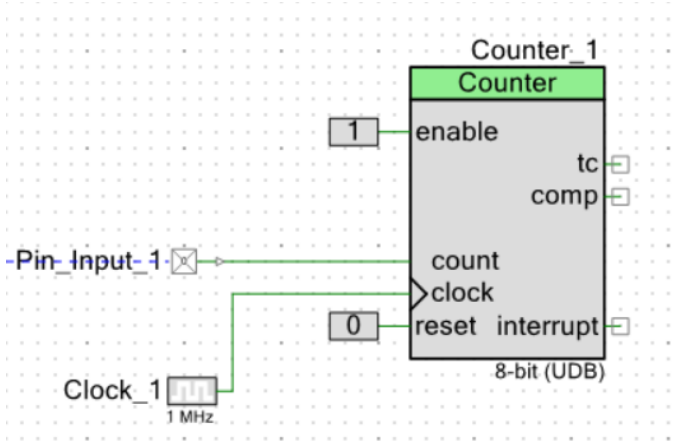
Period ~ $0 - 2^{\text{Bits}}$

Direction = Up or Down

Count-Edge = Fall or Rise

- The block increments the counter value upon detecting edges (rising or falling) on its count input
- Clock is used for sampling inputs to the counter (to avoid setup violation) > 2x (Signal Freq at Count)
- ‘tc’ goes high when counter value equals the period (or terminal count). It stays high for one clock cycle

Counter - Verilog

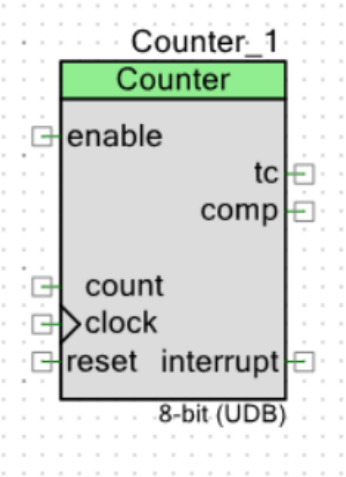



```
module counter (
    input clock,
    input reset,
    input count,
    input up,
    output reg [7:0] counter_reg
);

reg count_prev;

always @(posedge clock) begin
    count_prev ≤ count;

    if (count && !count_prev)
        counter_reg ≤ counter_reg + 1
end

endmodule
```

Bits = 8bit, Up Counter, Period ? Count-Edge ?

E.g., Detect Signal Freq

“Detect Frequency of an Unknown PWM Signal”



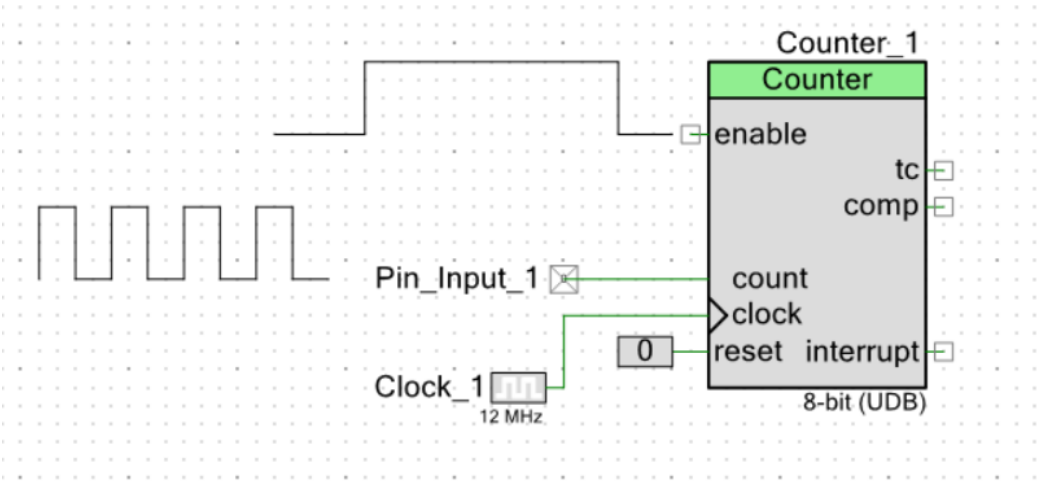
?

- The block increments the counter value upon detecting edges (rising or falling) on its count input
- Clock is used for sampling > 2x (Signal Freq at Count)

https://en.wikipedia.org/wiki/Pulse-width_modulation
Ref – creative commons

E.g., Detect Sig Freq (…)

“Detect Frequency of an Unknown Square Signal”



Enable Time ?

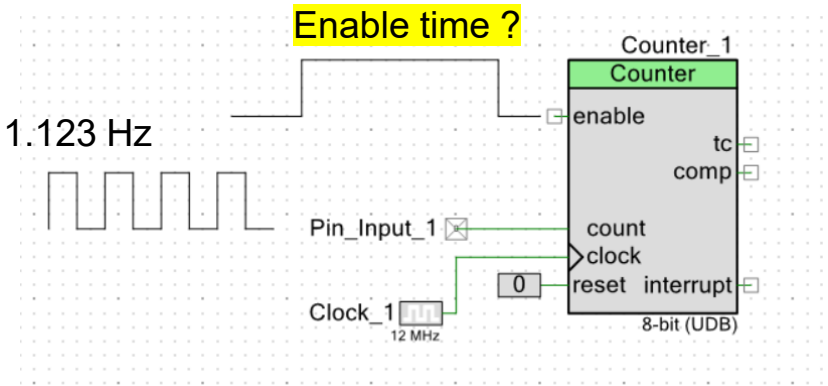
Period ?

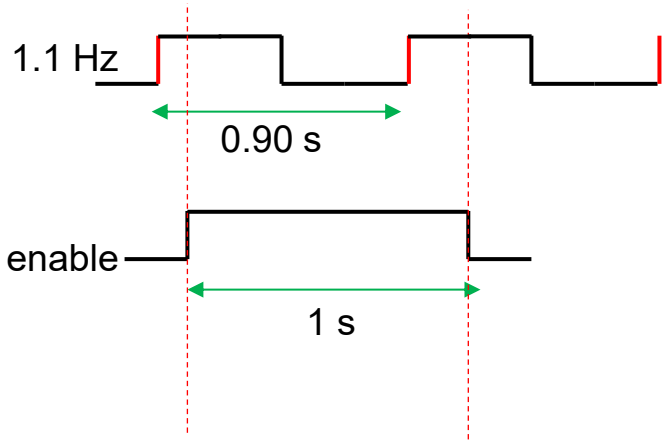
- The block increments the counter value upon detecting edges (rising or falling) on its count input
- Clock is used for sampling > 2x (Signal Freq at Count)

https://en.wikipedia.org/wiki/Pulse-width_modulation

E.g., Detect Sig Freq (…)

“Detect Frequency of an Unknown PWM Signal”

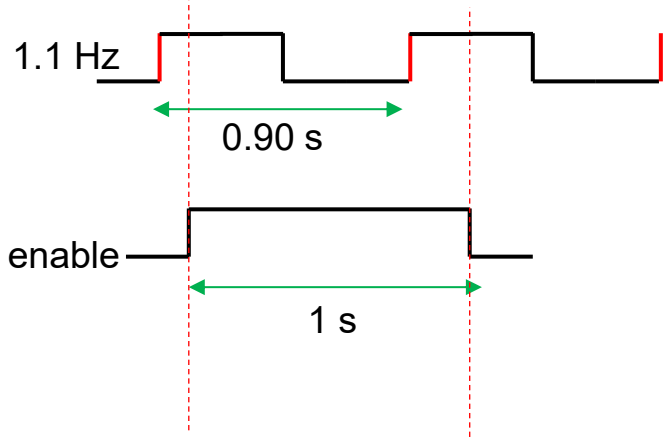




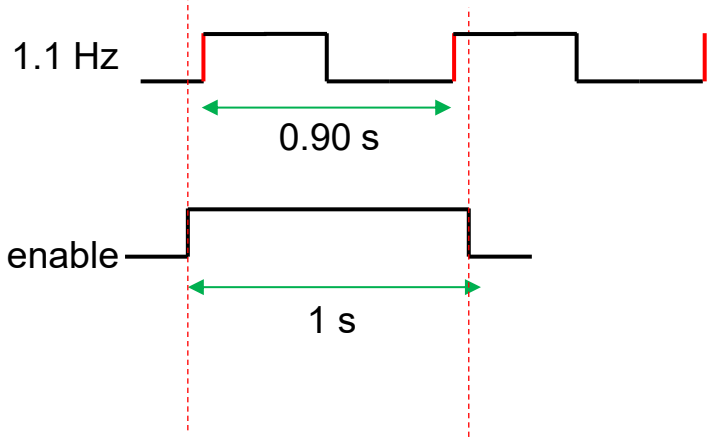
Counter-Value = 1

E.g., Detect Sig Freq (…)

“Detect Frequency of an Unknown PWM Signal”



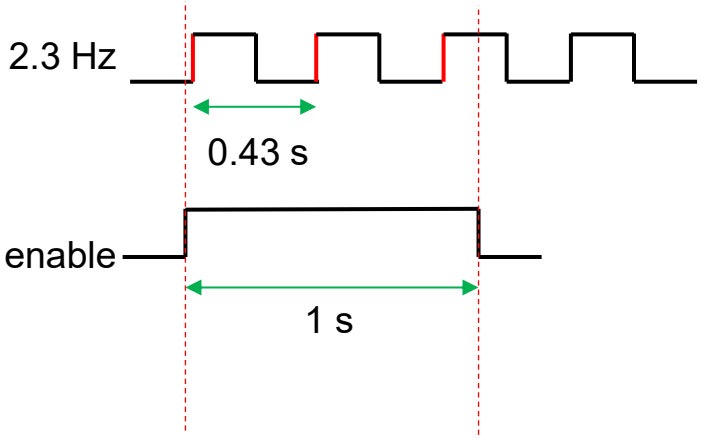
Counter-Value = 1 (i.e., 1Hz)



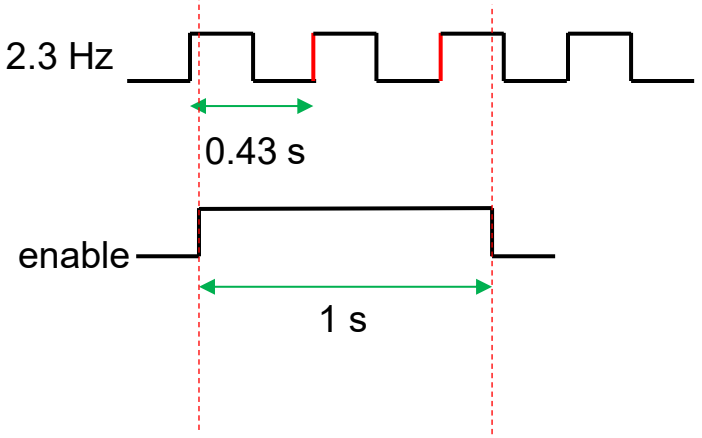
Counter-Value = 2 (i.e., 2Hz)

For a 1-second enable window, the counter can either report 1 or 2.

E.g., Detect Sig Freq (…)



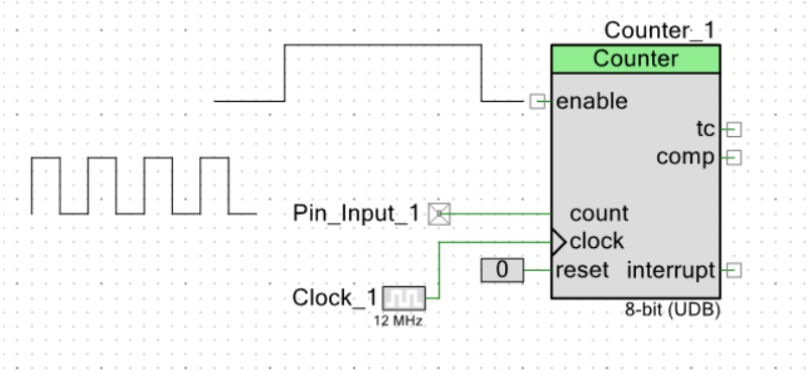
Counter-Value = 3 (3Hz)



Counter-Value = 2 (2Hz)

For a 1-second enable window, the counter can either report 2 or 3.

Resolution



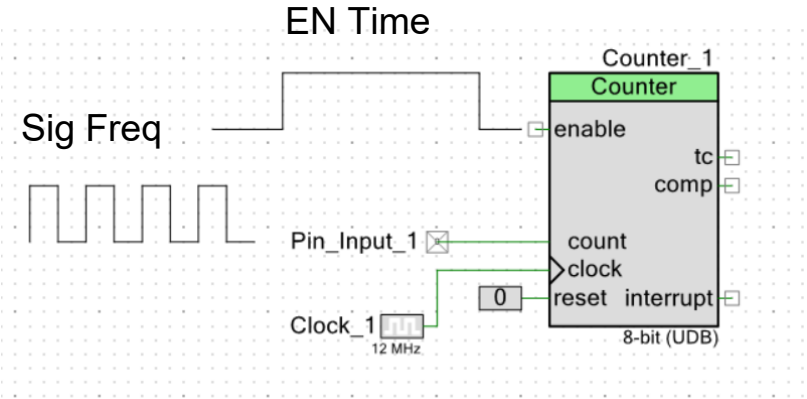
In this setting, the smallest change in the counter output is 1 count, corresponding to a frequency change of 1 Hz. Frequencies below 1 Hz cannot be accurately resolved using a single 1-second measurement window.

- For enable of 1 seconds, 1 count correspond to 1 Hz → Measurement Resolution = 1 Hz
- For enable of 5 seconds, 1 count correspond to 0.2 Hz → Measurement Resolution = 0.2 Hz
- For enable of 10 seconds, 1 count correspond to 0.1 Hz → Measurement Resolution = 0.1 Hz
- For enable of 100 seconds, 1 count correspond to 0.01 Hz → Measurement Resolution = 0.01 Hz

For a 5-second enable window, the minimum change in frequency corresponding to a one-count change is 0.2 Hz.

Question

- **EN Time = 1s, Sig Freq = 2.3456Hz, Counter Out ?**
- **EN Time = 1s, Sig Freq = 2.8456Hz, Counter Out ?**
- **EN Time = 5s, Sig Freq = 2.8456Hz, Counter Out ?**
- **EN Time = 10s, Sig Freq = 2.3456Hz, Counter Out ?**
- **EN Time = 100s, Sig Freq = 2.3456Hz, Counter Out ?**



Question

- **EN Time = 1s, Sig Freq = 2.3456Hz, Counter Out ?**

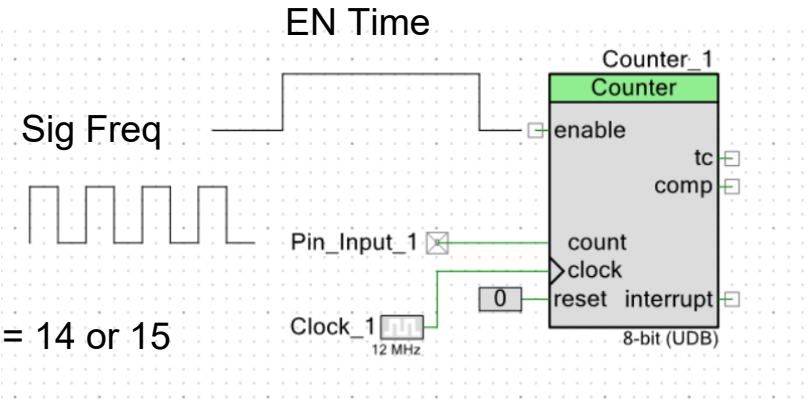
Number of cycles in 1 s = 2.3456 → Counter Out = 2 or 3

- **EN Time = 1s, Sig Freq = 2.8456Hz, Counter Out ?**

Number of cycles in 1 s = 2.8456 → Counter Out = 2 or 3

- **EN Time = 5s, Sig Freq = 2.8456Hz, Counter Out ?**

Number of cycles in 5 s = 5x2.8456 = 14.228 → Counter Out = 14 or 15



- **EN Time = 10s, Sig Freq = 2.3456Hz, Counter Out ?**

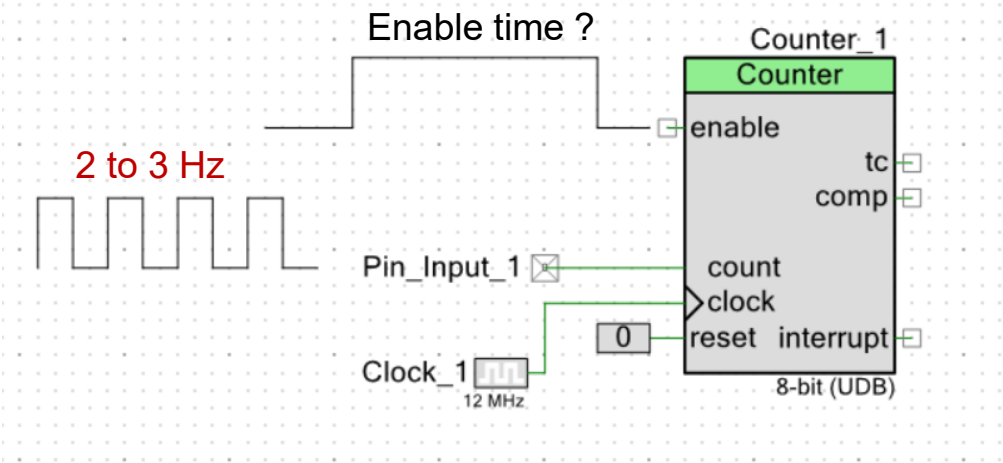
Number of cycles in 10 s = 23.456 → Counter Out = 23 or 24

- **EN Time = 100s, Sig Freq = 2.3456Hz, Counter Out ?**

Number of cycles in 100 s = 234.56 → Counter Out = 234 or 235

Question ?

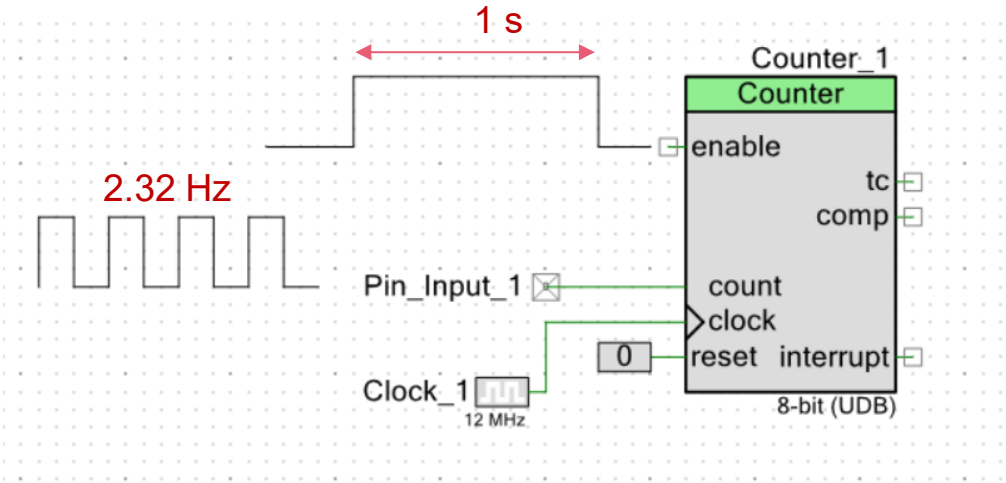
I need a freq resolution of **0.01 Hz** for a signal between **2 and 3 Hz.** Can this be achieved using an **8-bit counter** ? If so, how?



Resolution of 0.01 Hz require 100seconds of enable time ? A signal frequency in the range of 2 to 3Hz will technically require a counter which can count to ? (3x100=300)

Solution → Averaging

I need a freq resolution of **0.01 Hz** for a signal between **2 and 3 Hz.** Can this be achieved using an **8-bit counter** ? If so, how?



Solution: Use a 1-second enable time, repeat the measurement 100 times, and average the counter outputs to obtain a 0.01 Hz output resolution.

- Counter Outputs = 2 or 3
- 2 for 68% and 3 for 32%
- **Averaging → 2.32**

of Measurements Required ?